

what light means

digitalisierung und programmierung · input, session 4

agenda

- 1 agreement
- 2 counting information
- 3 your alphabet

agreement

what makes light
mean something?

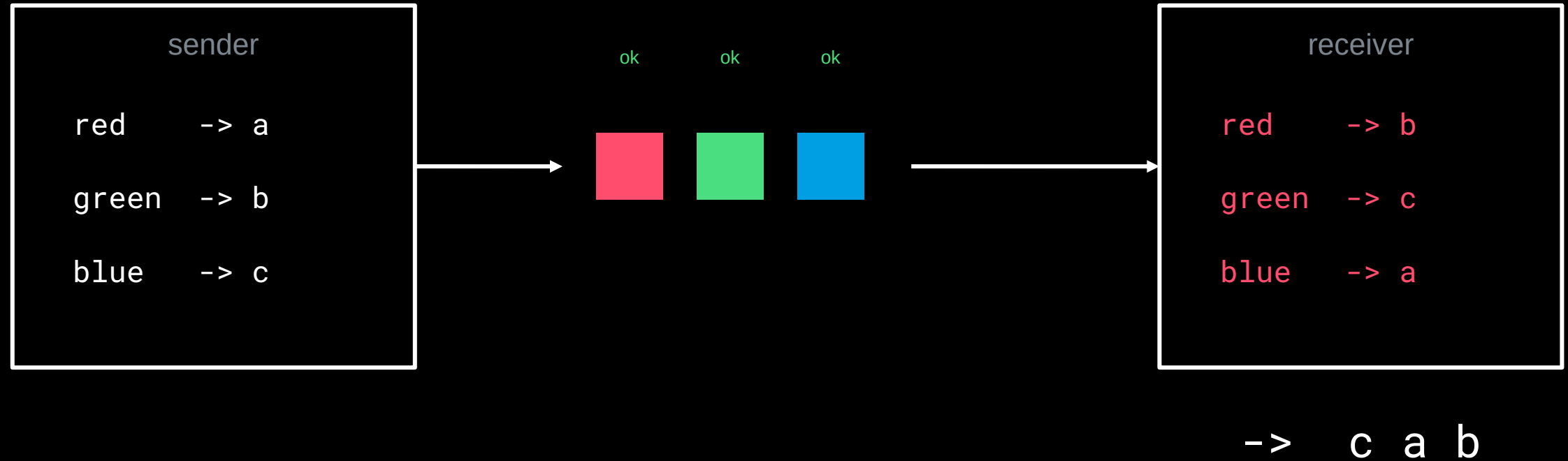


a symbol is an agreed, distinguishable state.

distinguishable: the receiver can tell it apart.

agreed: both sides gave it the same meaning.

perfect reception, perfect nonsense



the light was right. the agreement was not.

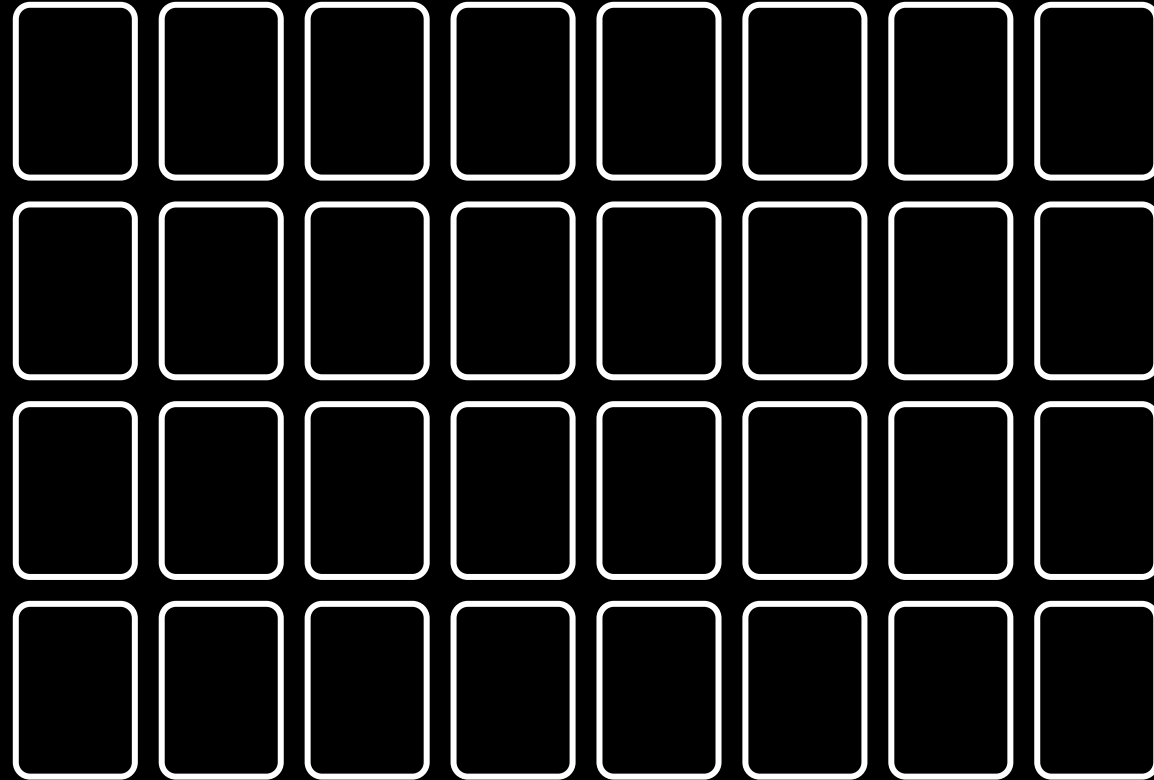
the agreement, written down

```
ALPHABET = {  
    "red":    "00",  
    "green":  "01",  
    "blue":   "10",  
    "yellow": "11",  
}
```

the receiver needs this mapping mirrored. one different line,
and clean reception turns to noise.

counting information

the guessing game, measured

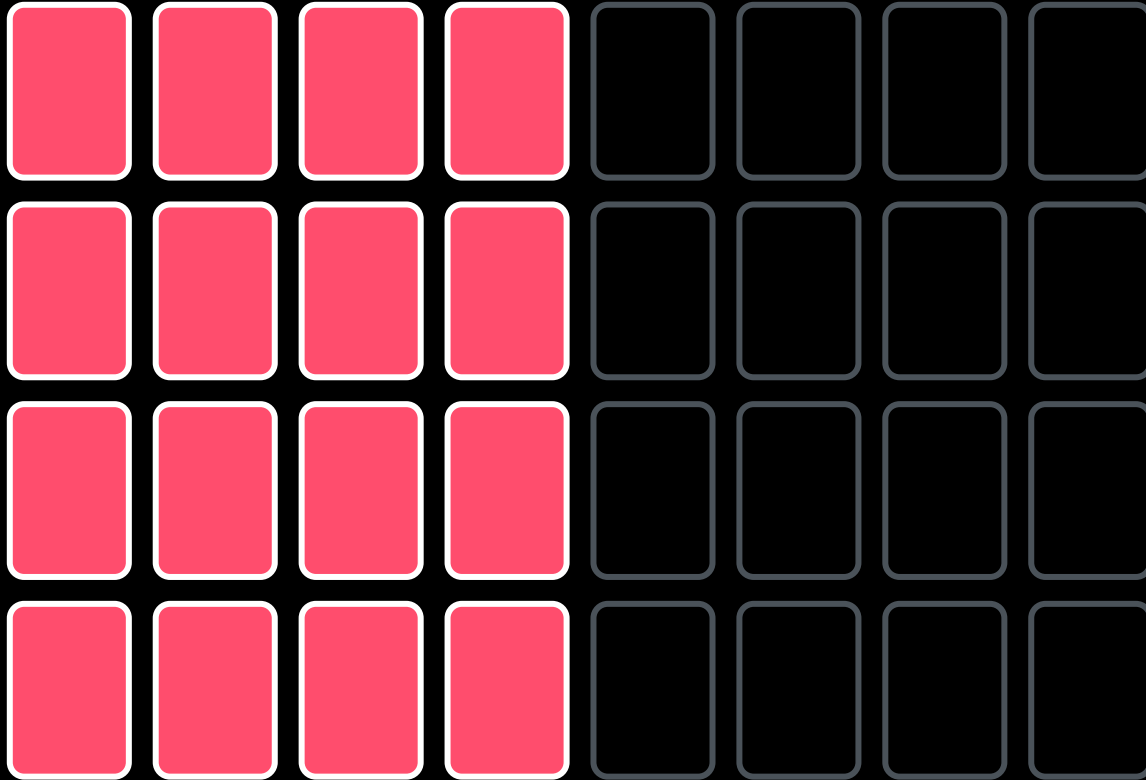


32 possibilities

that is the space you have to narrow down.

the right question

"is it red?" -> yes



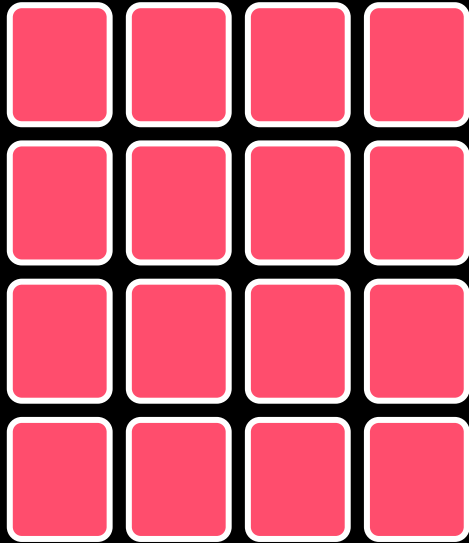
one answer:
16 possibilities gone.

"is it the ace of spades?"
almost always no,
almost nothing gone.

the best question splits what is left in half.

not every answer is worth one bit

"is it red?"



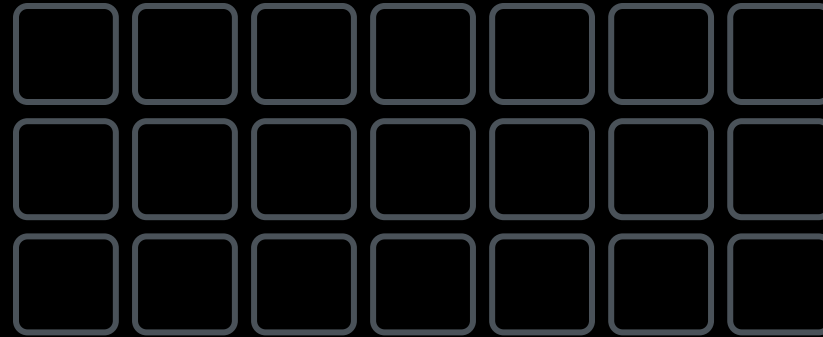
yes -> 16 left no -> 16 left

always exactly 1 bit

"is it the ace of spades?"



1 of 32



yes -> done no -> 31 left

5 bits, but almost never

the value of an answer depends on how surprising it is.

a rare answer tells you a lot.
an expected answer tells you little.

you cannot choose which answer you get.
so what counts is the average.

why halving wins on average

question	answer	chance	you learn	average
"is it red?"	yes	1 / 2	1 bit	
	no	1 / 2	1 bit	1.00 bit
"ace of spades?"	yes	1 / 32	5 bits	
	no	31 / 32	0.05 bits	0.20 bits

equal halves make every answer equally valuable.
that is what maximises the average.

one bit = one decision
between two possibilities.

a light switch holds one. no computer required.

counting what you learn

$$I = \log_2(N)$$

$$N = 2^I$$

N = number of possibilities

I = information in bits

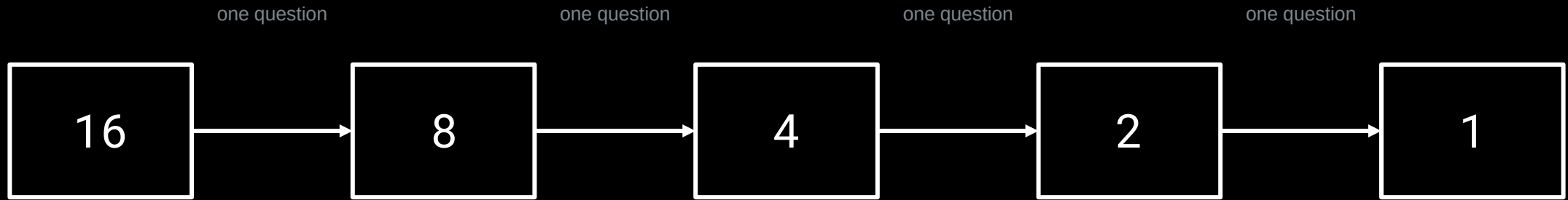
32 cards -> 5 bits

4 colours -> 2 bits

256 values -> 8 bits

the logarithm counts the halvings. it also measures your uncertainty: 32 possibilities = 5 bits of it.

counting bits by halving



16 symbols = 4 questions = 4 bits

4 symbols = 2 bits · 8 symbols = 3 bits

every doubling of the alphabet buys one more bit per symbol.

the payoff

same file, same symbol rate. bar length = time on the link.

2 colours · 1 bit per symbol



8 colours · 3 bits per symbol



same file, done



same symbol rate. a third of the time.

two symbols are enough for anything.
groups do the rest.

five two-way flashes make 32 combinations, room for a whole
alphabet of letters. cost: length, nothing else.

when symbols are not equally likely

e

•

most common letter

t

—

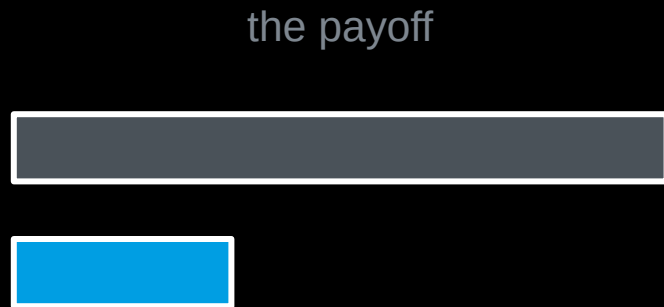
q

— — • —

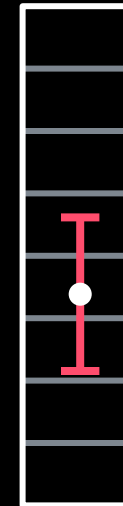
rare letter

if symbols are not equally likely,
give the common ones short codes.

the catch



the price



left: what a bigger alphabet buys.
right: what it costs.

your alphabet

today

- 1 agree on your alphabet, write it down
- 2 measure a profile for every symbol
- 3 fill `send_symbol()` and `receive_symbol()`
- 4 `run_test.py`: 50 tries, know your hit rate